Traffic Light Controller using Verilog

Name: Gautam Raj Chhabra
Project: Traffic Light Controller
Language: Verilog HDL
Tool Used: EDA Playground (Icarus Verilog)

1. Abstract

This project implements a Traffic Light Controller using a Moore Finite State Machine (FSM) in Verilog. The system controls traffic flow at an intersection between two directions: North-South and East-West. The controller cycles through four states to ensure safe and efficient traffic movement.

2. Introduction

Traffic light controllers are widely used in urban areas to regulate vehicle movement at intersections. Digital logic design and FSM techniques can be used to implement such controllers efficiently. In this project, a Moore FSM is used to design a traffic light system where outputs depend only on the current state.

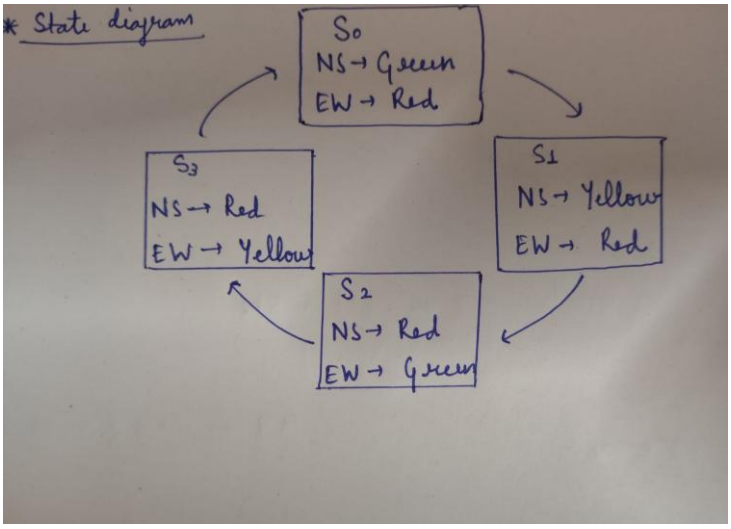
3. System Description

The system manages two directions of traffic: **North–South (NS)** and **East–West (EW)**.
Each direction has three lights: **Green, Yellow, and Red**.

4. State Diagram

The FSM consists of four states:

State	NS Light	EW Light
S0	Green	Red
S1	Yellow	Red
S2	Red	Green
S3	Red	Yellow



5. Verilog Design Code

```
module traffic_light_controller(
    input clk,
    input reset,

    output reg NS_G,
    output reg NS_Y,
    output reg NS_R,

    output reg EW_G,
    output reg EW_Y,
    output reg EW_R
);

reg [1:0] state, next_state;

parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10,
          S3 = 2'b11;



//// STATE REGISTER
always @(posedge clk or posedge reset)
begin
    if (reset)
        state <= S0;
```

```
    else
        state <= next_state;
end


//// NEXT STATE LOGIC
always @(*)
begin
    case(state)

        S0: next_state = S1;
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = S0;

        default: next_state = S0;

    endcase
end



//// OUTPUT LOGIC
always @(*)
begin
    case(state)

        S0: begin
            NS_G = 1;  NS_Y = 0;  NS_R = 0;
            EW_G = 0;  EW_Y = 0;  EW_R = 1;
        end

        S1: begin
            NS_G = 0;  NS_Y = 1;  NS_R = 0;
            EW_G = 0;  EW_Y = 0;  EW_R = 1;
        end

        S2: begin
            NS_G = 0;  NS_Y = 0;  NS_R = 1;
            EW_G = 1;  EW_Y = 0;  EW_R = 0;
        end

        S3: begin
            NS_G = 0;  NS_Y = 0;  NS_R = 1;
            EW_G = 0;  EW_Y = 1;  EW_R = 0;
        end
```

```
        default: begin
            NS_G = 0;  NS_Y = 0;  NS_R = 1;
            EW_G = 0;  EW_Y = 0;  EW_R = 1;
        end

    endcase
end

endmodule
```

6. Testbench

The testbench is used to verify the correct operation of the Traffic Light Controller. It generates a clock signal and applies a reset signal to initialize the system. The clock toggles periodically to simulate the operation of the controller over time. The $monitor command is used to continuously display the values of the traffic light signals during simulation. This allows observation of state transitions and helps verify that the traffic lights change in the correct sequence.

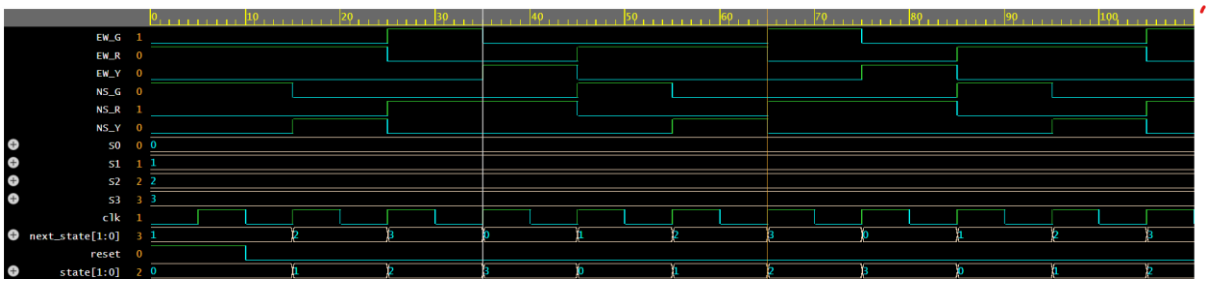
```
module tb;

reg clk;
reg reset;

wire NS_G, NS_Y, NS_R;
wire EW_G, EW_Y, EW_R;

traffic_light_controller DUT(clk, reset, NS_G, NS_Y, NS_R, EW_G,
EW_Y, EW_R);

initial
begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial
begin
```

```
    $monitor("time=%0t NS_G=%b NS_Y=%b NS_R=%b EW_G=%b EW_Y=%b
EW_R=%b", $time, NS_G, NS_Y, NS_R, EW_G, EW_Y, EW_R);

    reset = 1;
    #10;
    reset = 0;

    #100;
    $finish;
end

endmodule
```

7. Simulation Results

```
[2026-03-05 15:45:50 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
time=0 NS[G Y R]=1 0 0  EW[G Y R]=0 0 1
time=15 NS[G Y R]=0 1 0  EW[G Y R]=0 0 1
time=25 NS[G Y R]=0 0 1  EW[G Y R]=1 0 0
time=35 NS[G Y R]=0 0 1  EW[G Y R]=0 1 0
time=45 NS[G Y R]=1 0 0  EW[G Y R]=0 0 1
time=55 NS[G Y R]=0 1 0  EW[G Y R]=0 0 1
time=65 NS[G Y R]=0 0 1  EW[G Y R]=1 0 0
time=75 NS[G Y R]=0 0 1  EW[G Y R]=0 1 0
time=85 NS[G Y R]=1 0 0  EW[G Y R]=0 0 1
time=95 NS[G Y R]=0 1 0  EW[G Y R]=0 0 1
time=105 NS[G Y R]=0 0 1  EW[G Y R]=1 0 0
testbench.sv:35: $finish called at 110 (1s)
Done
```

8.EP Wave



9. Conclusion

The Traffic Light Controller was successfully implemented using a Moore FSM in Verilog. Simulation results confirm correct state transitions and proper traffic signal control. This project demonstrates the use of FSM-based design for real-world digital systems.